DAT151: Assignment 5 Report

Group: 5

Group Members: Soukup Jan, Fabienne Feilke

Date: April 12, 2026

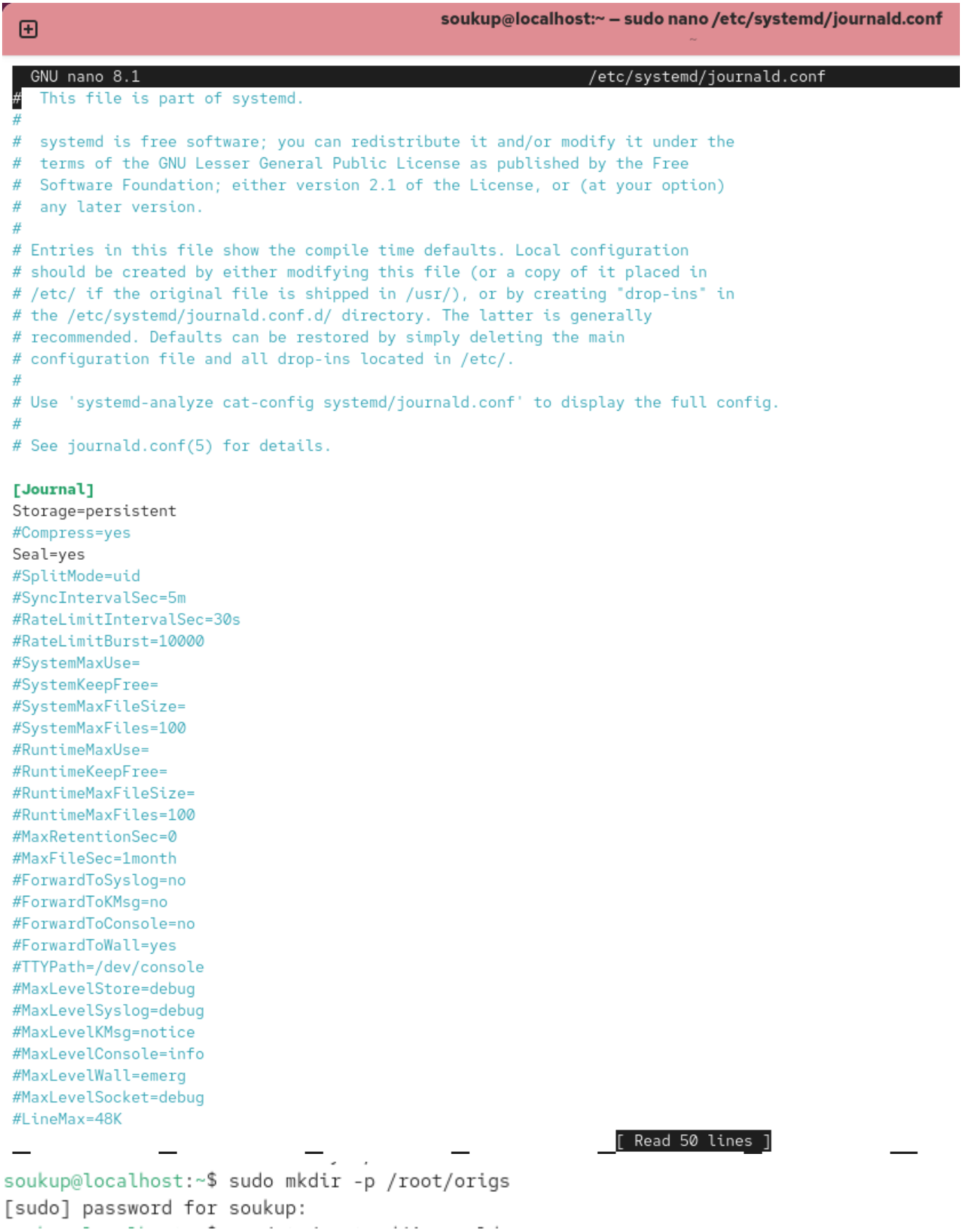
Task 1: Logging

a) Configure systemd journal

Configure systemd journal at the lab to save the logs persistent in the file system.

The screenshots for this step show the journald configuration workflow: copying the default file to `/etc/systemd/journald.conf`, creating a backup copy in `/root/origs/`, and editing the config file.

```
soukup@localhost:~$ sudo cp /usr/lib/systemd/journald.conf /etc/systemd/journald.conf
soukup@localhost:~$ sudo cp /etc/systemd/journald.conf /root/origs/journald.conf.orig
soukup@localhost:~$ sudo nano /etc/systemd/journald.conf
```




```
soukup@localhost:~$ sudo mkdir -p /root/origs
[sudo] password for soukup:
```

The configuration was applied by copying the default file, editing it, and restarting the service. After that, `journalctl --flush` was used so the current journal was written into the persistent directory, and `ls -la /var/log/journal/` confirmed that the journal directory existed.

```
sudo cp /usr/lib/systemd/journald.conf /etc/systemd/journald.conf
sudo nano /etc/systemd/journald.conf
sudo systemctl restart systemd-journald
sudo journalctl --flush
sudo ls -la /var/log/journal/
```

```
soukup@localhost:~$ sudo mkdir -p /var/log/journal
soukup@localhost:~$ sudo systemd-tmpfiles --create --prefix /var/log/journal
soukup@localhost:~$ sudo systemctl restart systemd-journald
soukup@localhost:~$ sudo journalctl --flush
soukup@localhost:~$ sudo systemctl restart systemd-journald
soukup@localhost:~$ sudo ls -la /var/log/journal/
total 8
drwxr-sr-x+  3 root systemd-journal   46 Mar 16 12:21 .
drwxr-xr-x. 17 root root            4096 Mar 16 12:17 ..
drwxr-sr-x+  2 root systemd-journal 4096 Mar 16 12:21 290f7160f81148eba941e54c910e6caa
soukup@localhost:~$
```

This means the system will keep journal entries across reboots instead of storing them only in memory.

b) Protect journal logs

Protect the journal logs from unnoticed alteration by enabling the Seal feature and create a sealing key. Verify the logs and prove that the logs are sealed. The verification output will show a time interval that is not sealed.

The screenshot below shows the sealing setup and verification step. After restarting `systemd-journald`, `journalctl --setup-keys` was run to generate the sealing key, and `journalctl --verify` was used to check the journal files.

```
soukup@localhost:~$ sudo systemctl restart systemd-journald
soukup@localhost:~$ sudo journalctl --setup-keys
Compiled without forward-secure sealing support.
soukup@localhost:~$ sudo journalctl --verify
PASS: /run/log/journal/290f7160f81148eba941e54c910e6caa/system@07a5a01f2fb244cb8d2bb8282fbe05b3-0000000000000001-00064d22e04361e6.journal
PASS: /run/log/journal/290f7160f81148eba941e54c910e6caa/system.journal
```

```
sudo systemctl restart systemd-journald
sudo journalctl --setup-keys
sudo journalctl --verify
```

The output shows a warning that the build was compiled without forward-secure sealing support, but the verification still reports `PASS` for the journal files that were checked. That is the verification output used to demonstrate the integrity of the stored journal.

c) Syslog local5 facility

The syslog facility names “local0” to “local7” are used for local custom messages defined by an administrator. Create your own configuration that saves only the critical level and information level messages from the local5 facility to a file local5 in the /var/log directory. Test your configuration using the logger command with specific

messages. Look under the /var/log directory and verify that the messages you added are saved at the correct location.

The screenshots show creating `/etc/rsyslog.d/local5.conf` and editing the file with a custom rule. The rule writes only `local5.info` and `local5.crit` messages to `/var/log/local5`.

```
soukup@localhost:~$ sudo nano /etc/rsyslog.d/local5.conf
soukup@localhost:~$ sudo systemctl restart rsyslog
```


```
sudo nano /etc/rsyslog.d/local5.conf
sudo systemctl restart rsyslog
logger -p local5.info "Task 1c: This is an INFO message."
logger -p local5.crit "Task 1c: This is a CRITICAL message."
logger -p local5.err "Task 1c: This is an ERROR message and should be
ignored."
sudo cat /var/log/local5
```

```
soukup@localhost:~$ logger -p local5.info "Task 1c: This is an INFO message."
soukup@localhost:~$ logger -p local5.crit "Task 1c: This is a CRITICAL message."
soukup@localhost:~$ logger -p local5.err "Task 1c: This is an ERROR message and should be ignored."
soukup@localhost:~$ sudo cat /var/log/local5
Mar 16 13:36:39 localhost soukup[10243]: Task 1c: This is an INFO message.
Mar 16 13:36:43 localhost soukup[10249]: Task 1c: This is a CRITICAL message.
soukup@localhost:~$ sudo cat /var/log/local5
Mar 16 13:36:39 localhost soukup[10243]: Task 1c: This is an INFO message.
Mar 16 13:36:43 localhost soukup[10249]: Task 1c: This is a CRITICAL message.
soukup@localhost:~$
```

The configuration line `local5.=info;local5.=crit /var/log/local5` means that only messages from facility `local5` with severity `info` or `crit` are stored in that file. The `err` message does not match the rule and is therefore not written there. The screenshot of `/var/log/local5` shows the `INFO` and `CRITICAL` messages in the correct location.

Task 2: LDAP

In this task you will set up an LDAP server and client(s). Use one of the computers of the group as the LDAP server, and the others as clients.

Installation and Configuration

Document your configurations clearly, include your LDIF files in your report, and explain how user records were added to your server.

We installed OpenLDAP server/client packages.

```
soukup@localhost:~$ sudo dnf install openldap-clients openldap-servers
```

Then we enabled and verified `slapd`.

```
soukup@localhost:~$ sudo systemctl enable --now slapd
Created symlink '/etc/systemd/system/openldap.service' → '/usr/lib/systemd/system/slapd.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/slapd.service' → '/usr/lib/systemd/system/slapd.service'
.
soukup@localhost:~$ sudo systemctl status slapd
● slapd.service - OpenLDAP Server Daemon
     Loaded: loaded (/usr/lib/systemd/system/slapd.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-03-23 12:24:59 CET; 15s ago
 Invocation: f6d48d53e64c428d8b9b1a38326d1c68
       Docs: man:slapd
             man:slapd-config
             man:slapd-mdb
             file:///usr/share/doc/openldap-servers/guide.html
    Process: 5374 ExecStartPre=/usr/libexec/openldap/check-config.sh (code=exited, status=0/SUCCESS)
    Process: 5395 ExecStart=/usr/sbin/slapd -u ldap -h ldap:/// ldaps:/// ldapi:/// (code=exited, status=0/SUCCESS)
   Main PID: 5396 (slapd)
      Tasks: 2 (limit: 46648)
     Memory: 3.5M (peak: 5.3M)
        CPU: 74ms
     CGroup: /system.slice/slapd.service
             └─5396 /usr/sbin/slapd -u ldap -h "ldap:/// ldaps:/// ldapi:///"

Mar 23 12:24:59 localhost.localdomain systemd[1]: Starting slapd.service - OpenLDAP Server Daemon...
Mar 23 12:24:59 localhost.localdomain runuser[5377]: pam_unix(runuser:session): session opened for user ldap(uid=55)>
Mar 23 12:24:59 localhost.localdomain runuser[5377]: pam_unix(runuser:session): session closed for user ldap
Mar 23 12:24:59 localhost.localdomain slapd[5395]: @(#) $OpenLDAP: slapd 2.6.9 (Aug 28 2025 00:00:00) $
                                                    openldap
Mar 23 12:24:59 localhost.localdomain slapd[5396]: slapd starting
Mar 23 12:24:59 localhost.localdomain systemd[1]: Started slapd.service - OpenLDAP Server Daemon.
soukup@localhost:~$
```

```
sudo dnf install openldap-clients openldap-servers
sudo systemctl enable --now slapd
sudo systemctl status slapd
```

`openldap-clients` provides the LDAP client tools such as `ldapadd`, `ldapmodify`, and `ldapsearch`, while `openldap-servers` installs the server daemon `slapd`. Enabling the service makes the LDAP server start automatically on boot.

DN Suffix and Manager Password

Configure the database with the new DNs (base DN and administrator DN) and set up the manager password and access.

Base DN LDIF (`basedn.ldif`):

```
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcSuffix
olcSuffix: dc=h68,dc=dat151
replace: olcRootDN
olcRootDN: cn=Manager,dc=h68,dc=dat151
```

Manager Config LDIF (`manager.ldif`):

```
dn: olcDatabase={2}mdb,cn=config
changetype: modify
add: olcRootPW
olcRootPW: {SSHA}<hash generated with slappasswd>
add: olcAccess
olcAccess: to * by dn="cn=Manager,dc=h68,dc=dat151" write by self write by
* read
```

First we create `basedn.ldif`, run `ldapmodify`, and inspect the initial error.

```
soukup@localhost:~$ mkdir task5
soukup@localhost:~$ cd task5/
soukup@localhost:~/task5$ nano basedn.ldif
soukup@localhost:~/task5$ sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f basedn.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
ldapmodify: wrong attributeType at line 5, entry "olcDatabase={2}mdb,cn=config"
soukup@localhost:~/task5$ sudo systemctl restart slapd
sudo systemctl status slapd
● slapd.service - OpenLDAP Server Daemon
     Loaded: loaded (/usr/lib/systemd/system/slapd.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-03-23 12:27:14 CET; 43ms ago
 Invocation: 2de53747923d4e8889740a34bf38c505
       Docs: man:slapd
             man:slapd-config
             man:slapd-mdb
             file:///usr/share/doc/openldap-servers/guide.html
    Process: 5728 ExecStartPre=/usr/libexec/openldap/check-config.sh (code=exited, status=0/SUCCESS)
    Process: 5749 ExecStart=/usr/sbin/slapd -u ldap -h ldap:/// ldaps:/// ldapi:/// (code=exited, status=0/SUCCESS)
   Main PID: 5750 (slapd)
      Tasks: 2 (limit: 46648)
     Memory: 3.4M (peak: 5.4M)
        CPU: 63ms
     CGroup: /system.slice/slapd.service
             └─5750 /usr/sbin/slapd -u ldap -h "ldap:/// ldaps:/// ldapi:///"

Mar 23 12:27:14 localhost.localdomain systemd[1]: Starting slapd.service - OpenLDAP Server Daemon...
Mar 23 12:27:14 localhost.localdomain runuser[5731]: pam_unix(runuser:session): session opened for user ldap(uid=55)
Mar 23 12:27:14 localhost.localdomain runuser[5731]: pam_unix(runuser:session): session closed for user ldap
Mar 23 12:27:14 localhost.localdomain slapd[5749]: @(#) $OpenLDAP: slapd 2.6.9 (Aug 28 2025 00:00:00) $
                                                   openldap
Mar 23 12:27:14 localhost.localdomain slapd[5750]: slapd starting
Mar 23 12:27:14 localhost.localdomain systemd[1]: Started slapd.service - OpenLDAP Server Daemon.
lines 1-24/24 (END)
```

```
soukup@localhost:~/task5 — nano basedn.ldif
~/task5

  GNU nano 8.1                                  basedn.ldif
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcSuffix
olcSuffix: dc=h68,dc=dat151
replace: olcRootDN
olcRootDN: cn=Manager,dc=h68,dc=dat151
```

Then we generate manager password hash and draft `manager.ldif`.

```
soukup@localhost:~/task5$ slappasswd
New password:
Re-enter new password:
{SSHA}pZnAXmkpKFctPBgN5jo4JQ/m0yb9skgS
```

soukup@localhost:~/task5 — nano manager.ldif
~/task5

```
 GNU nano 8.1                          manager.ldif                          Modified
dn: olcDatabase={2}mdb,cn=config
changetype: modify
add: olcRootPW
olcRootPW: <password hash>
add: olcAccess
olcAccess: to * by dn="cn=Manager,dc=h68,dc=dat151" write by self write by * read
```

Then we fix LDIF separators/format and apply updated suffix/root DN.

```
soukup@localhost:~/task5$ sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f manager.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
ldapmodify: wrong attributeType at line 5, entry "olcDatabase={2}mdb,cn=config"
```

soukup@localhost:~/task5 — nano basedn.ldif
~/task5

```
 GNU nano 8.1                                basedn.ldif
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcSuffix
olcSuffix: dc=h68,dc=dat151
-
replace: olcRootDN
olcRootDN: cn=Manager,dc=h68,dc=dat151
```

```
soukup@localhost:~/task5$ nano basedn.ldif
soukup@localhost:~/task5$ sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f basedn.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
modifying entry "olcDatabase={2}mdb,cn=config"
```

We apply manager permissions and password, then replace root password using `fixpw.ldif`.

soukup@localhost:~/task5 — nano manager.ldif
~/task5

```
 GNU nano 8.1                                  manager.ldif
dn: olcDatabase={2}mdb,cn=config
changetype: modify
add: olcRootPW
olcRootPW: <password hash>
-
add: olcAccess
olcAccess: to * by dn="cn=Manager,dc=h68,dc=dat151" write by self write by * read
```

soukup@localhost:~/task5 — nano manager.ldif
~/task5

```
 GNU nano 8.1                                  manager.ldif
dn: olcDatabase={2}mdb,cn=config
changetype: modify
add: olcRootPW
olcRootPW: pZnAXmkpKFctPBgN5jo4JQ/m0yb9skgS
-
add: olcAccess
olcAccess: to * by dn="cn=Manager,dc=h68,dc=dat151" write by self write by * read
```

```
soukup@localhost:~/task5$ nano manager.ldif
soukup@localhost:~/task5$ sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f manager.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
modifying entry "olcDatabase={2}mdb,cn=config"
```

```
soukup@localhost:~/task5$ slappasswd
New password:
Re-enter new password:
{SSHA}lFdGdlsZBjR9yBKjHN+6citudRqQ1ObP
soukup@localhost:~/task5$
```

soukup@localhost:~/task5 — nano fixpw.ldif
~/task5

```
 GNU nano 8.1                                    fixpw.ldif
dn: olcDatabase={2}mdb,cn=config
changetype: modify
replace: olcRootPW
olcRootPW: {SSHA}lFdGdlsZBjR9yBKjHN+6citudRqQ1ObP
```

```
soukup@localhost:~/task5$ nano fixpw.ldif
soukup@localhost:~/task5$ sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f fixpw.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
modifying entry "olcDatabase={2}mdb,cn=config"

soukup@localhost:~/task5$ sudo ldapadd -D "cn=Manager,dc=h68,dc=dat151" -x -W -f top.ldif
Enter LDAP Password:
ldap_bind: Invalid credentials (49)
soukup@localhost:~/task5$ sudo ldapadd -D "cn=Manager,dc=h68,dc=dat151" -x -W -f top.ldif
Enter LDAP Password:
adding new entry "dc=h68,dc=dat151"
```

The changes were applied with `ldapmodify -Y EXTERNAL -H ldapi:/// -f basedn.ldif` and `ldapmodify -Y EXTERNAL -H ldapi:/// -f manager.ldif`. This updates the live OpenLDAP configuration database under `cn=config` without editing the backend files by hand.

Schemas

Install necessary schemas (COSINE, NIS).

We run COSINE/NIS schema add commands (output shows duplicate attributes, i.e., already loaded).

```
soukup@localhost:~/task5$ sudo ldapadd -Y EXTERNAL -H ldapi:/// -f /etc/openldap/schema/cosine.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
adding new entry "cn=cosine,cn=schema,cn=config"
ldap_add: Other (e.g., implementation specific) error (80)
        additional info: olcAttributeTypes: Duplicate attributeType: "0.9.2342.19200300.100.1.2"

soukup@localhost:~/task5$ sudo ldapadd -Y EXTERNAL -H ldapi:/// -f /etc/openldap/schema/nis.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
adding new entry "cn=nis,cn=schema,cn=config"
ldap_add: Other (e.g., implementation specific) error (80)
        additional info: olcAttributeTypes: Duplicate attributeType: "1.3.6.1.1.1.1.2"
```

Then we verify schema entries with `ldapsearch`.

```
soukup@localhost:~/task5$ sudo ldapsearch -LLL -Y EXTERNAL -H ldapi:/// -b "cn=schema,cn=config" cn
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
dn: cn=schema,cn=config
cn: schema

dn: cn={0}core,cn=schema,cn=config
cn: {0}core

dn: cn={1}cosine,cn=schema,cn=config
cn: {1}cosine

dn: cn={2}nis,cn=schema,cn=config
cn: {2}nis
```

After that we add `inetorgperson` schema required for user object classes.

```
soukup@localhost:~/task5$ sudo ldapadd -Y EXTERNAL -H ldapi:/// -f /etc/openldap/schema/inetorgperson.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
adding new entry "cn=inetorgperson,cn=schema,cn=config"

soukup@localhost:~/task5$ nano justuser.ldif
```

```
sudo ldapadd -Y EXTERNAL -H ldapi:/// -f /etc/openldap/schema/cosine.ldif
sudo ldapadd -Y EXTERNAL -H ldapi:/// -f /etc/openldap/schema/nis.ldif
sudo ldapsearch -LLL -Y EXTERNAL -H ldapi:/// -b "cn=
{2}nis,cn=schema,cn=config"
```

The `ldapsearch` output shows that the schema entries are present in the configuration database, including `core`, `cosine`, and `nis`.

Create and Fill User Records Database

Create the database for user records and fill it with users and groups.

Database DN LDIF (`top.ldif`):

```
dn: dc=h68,dc=dat151
dc: h68
objectClass: top
objectClass: domain
```

User/Group Creation: The database was filled in two steps. First, the organizational units for groups and people were created, and then the actual user entry was added under `ou=People`.

Organizational Units (`ou.ldif`):

```
dn: ou=Group,dc=h68,dc=dat151
ou: Group
objectClass: organizationalUnit

dn: ou=People,dc=h68,dc=dat151
ou: People
objectClass: organizationalUnit
```

User Entry (`justuser.ldif`):

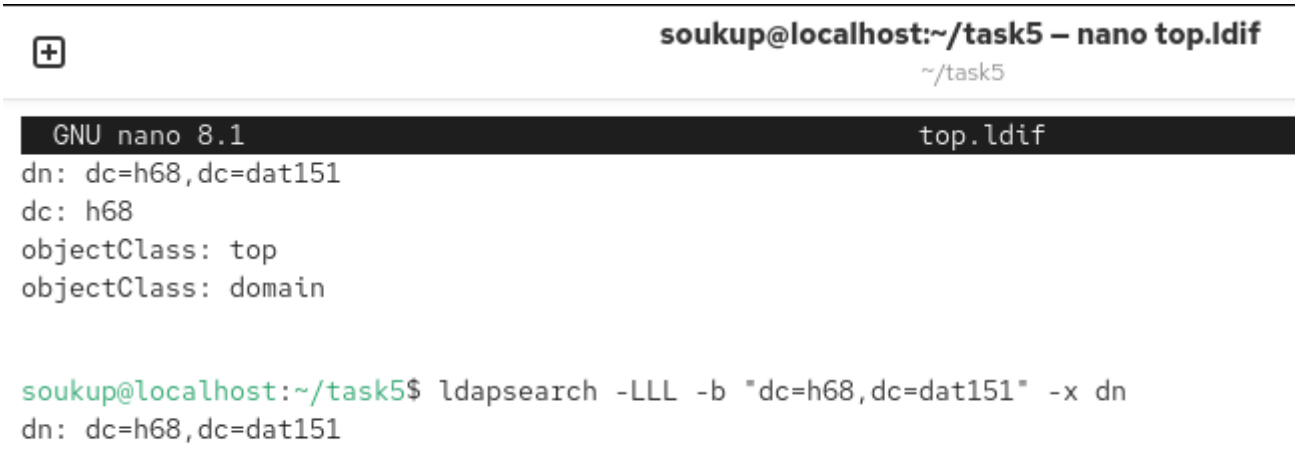
```
dn: uid=testuser,ou=People,dc=h68,dc=dat151
objectClass: inetOrgPerson
objectClass: posixAccount
```

```
objectClass: shadowAccount
uid: testuser
sn: User
givenName: Test
cn: Test User
displayName: Test User
uidNumber: 2000
gidNumber: 2000
userPassword: {SSHA}<hash generated with slappasswd>
gecos: Test User
loginShell: /bin/bash
homeDirectory: /home/testuser
```

Group Entry:

```
dn: cn=testgroup,ou=Group,dc=h68,dc=dat151
objectClass: posixGroup
cn: testgroup
gidNumber: 2000
```

We prepare database entry and verify base DN query.



We set LDAP client defaults and create organizational units.

We first attempted combined `userdata.ldif`, then switched to corrected `justuser.ldif`.

soukup@localhost:~/task5 — nano userdata.ldif

~/task5

```
  GNU nano 8.1                                  userdata.ldif
# Create a Group for the user
dn: cn=testgroup,ou=Group,dc=h68,dc=dat151
objectClass: posixGroup
cn: testgroup
gidNumber: 2000

# Create the User
dn: uid=testuser,ou=People,dc=h68,dc=dat151
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: testuser
sn: User
givenName: Test
cn: Test User
displayName: Test User
uidNumber: 2000
gidNumber: 2000
userPassword: {SSHA}Mbp2jNUtn+9LpCSZ70Qpzj94LeXJUvk+
gecos: Test User
loginShell: /bin/bash
homeDirectory: /home/testuser


soukup@localhost:~/task5$ slappasswd
New password:
Re-enter new password:
{SSHA}Mbp2jNUtn+9LpCSZ70Qpzj94LeXJUvk+
soukup@localhost:~/task5$ nano userdata.ldif

soukup@localhost:~/task5$ ldapadd -D "cn=Manager,dc=h68,dc=dat151" -x -W -f userdata.ldif
Enter LDAP Password:
adding new entry "cn=testgroup,ou=Group,dc=h68,dc=dat151"

adding new entry "uid=testuser,ou=People,dc=h68,dc=dat151"
ldap_add: Invalid syntax (21)
        additional info: objectClass: value #0 invalid per syntax
```

soukup@localhost:~/task5 — nano justuser.ldif
~/task5

```
 GNU nano 8.1                                    justuser.ldif
dn: uid=testuser,ou=People,dc=h68,dc=dat151
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: testuser
sn: User
givenName: Test
cn: Test User
displayName: Test User
uidNumber: 2000
gidNumber: 2000
userPassword: {SSHA}Mbp2jNUtn+9LpCSZ70Qpzj94LeXJUvk+
gecos: Test User
loginShell: /bin/bash
homeDirectory: /home/testuser
```

```
soukup@localhost:~/task5$ ldapadd -D "cn=Manager,dc=h68,dc=dat151" -x -W -f justuser.ldif
Enter LDAP Password:
adding new entry "uid=testuser,ou=People,dc=h68,dc=dat151"
```

Finally we opened the LDAP service in firewall.

```
soukup@localhost:~/task5$ sudo firewall-cmd --add-service=ldap --permanent
success
soukup@localhost:~/task5$ sudo firewall-cmd --reload
success
soukup@localhost:~/task5$
```

```
sudo ldapadd -D "cn=Manager,dc=h68,dc=dat151" -x -W -f top.ldif
sudo ldapadd -D "cn=Manager,dc=h68,dc=dat151" -x -W -f ou.ldif
sudo ldapadd -D "cn=Manager,dc=h68,dc=dat151" -x -W -f justuser.ldif
sudo ldapsearch -LLL -x -b "dc=h68,dc=dat151" dn
```

The final `ldapsearch` verification confirms that the base DN, both organizational units, and the user record exist in the directory.

Task 3: SSSD

Configure SSSD to use the LDAP from Task 2 for user authentication. You will also configure LDAPS (LDAP over SSL/TLS).

SSSD Configuration

Configure `/etc/sssd/sssd.conf`, use `authselect` to select the PAM profile for SSSD, and enable `with-mkhomedir`.

We installed the oddjob package.

```
soukup@localhost:~/task5$ sudo dnf install sssd sssd-ldap oddjob-mkhomedir authselect
```

- oddjob-mkhomedir automatically creates /home/username when an LDAP user logs in for the first time
- authselect rewires our systems PAM to point towards SSSD

At first we created the config file on the client.

```
ffeilke@host22:~$ sudo chown root:root /etc/sssd/sssd.conf
ffeilke@host22:~$ sudo chmod 600 /etc/sssd/sssd.conf
ffeilke@host22:~$ sudo systemctl enable --now sssd
```

We set file permissions so that only the root user can read or write to the file.

```
 GNU nano 8.1                                    /etc/sssd/sssd.conf
[sssd]
services = nss, pam
domains = LDAP

[domain/LDAP]
id_provider = ldap
auth_provider = ldap
ldap_uri = ldap://10.0.0.68
ldap_search_base = dc=h68,dc=dat151
# Temporary debug setting for Task 3 initial testing
ldap_auth_disable_tls_never_use_in_production = true
ldap_id_use_start_tls = false
cache_credentials = true
enumerate = true

# Required if using shadowAccount in LDAP
ldap_chpass_update_last_change = true
```

Login Verification (Client)

The client computer should be a different computer than the LDAP host, and the user should not exist on the client prior to log in. You must must a add screen shot to the report of a working log in on the client: `su - <some_user>` (To be run on the client, NOT the LDAP host).

Screenshot:

```
ffeilke@host22:~$ sudo nano /etc/hosts
ffeilke@host22:~$ sudo mkdir -p /etc/openldap/certs
[sudo] password for ffeilke:
ffeilke@host22:~$ sudo nano /etc/sssd/sssd.conf
ffeilke@host22:~$ sudo sss_cache -E
ffeilke@host22:~$ sudo systemctl restart sssd
ffeilke@host22:~$ su - testuser
Password:
Last login: Mon Mar 23 13:07:45 CET 2026 on pts/0
testuser@host67:~$ exit
logout
ffeilke@host22:~$
```

LDAP Access Control

With login working from the client using SSSD, the LDAP access rules should be restricted further by modifying the LDAP `olcAccess` parameter. See e.g. the man pages slapd-config(5) and slapd.access(5).

```
soukup@localhost:~/task5$ nano access.ldif
soukup@localhost:~/task5$ sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f access.ldif
[sudo] password for soukup:
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
modifying entry "olcDatabase={2}mdb,cn=config"
```

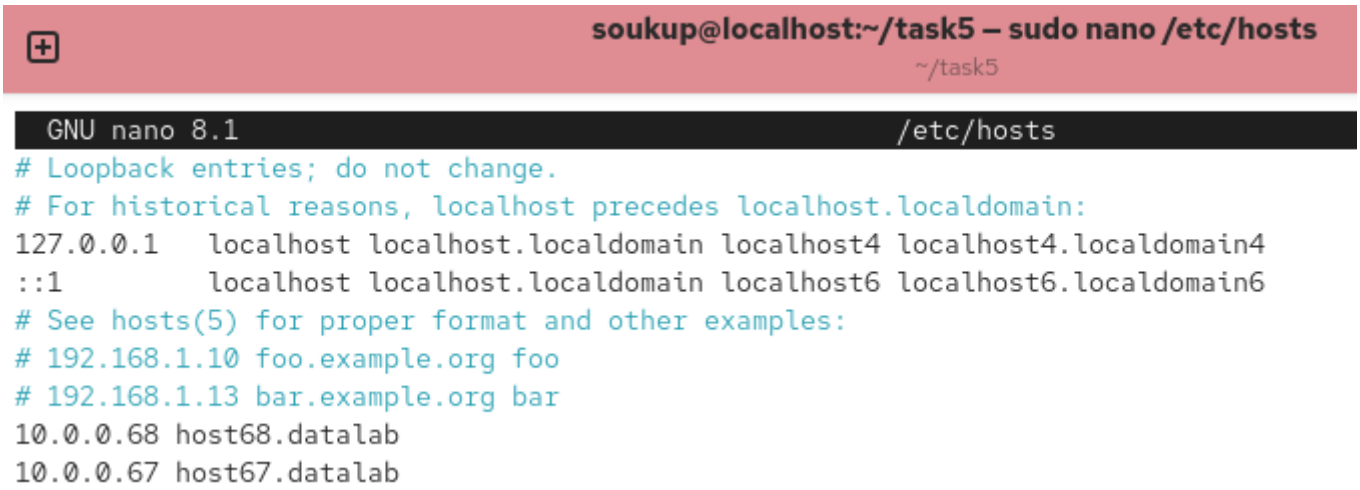
LDAPS Configuration

Configure the LDAP server to use the encrypted `ldaps` channel. Create/Sign certificates, configure standard hostname resolution (/etc/hosts), and update LDAP configuration.

Certificate Setup: We configurated the LDAP Server Configuration as follows:

```
 GNU nano 8.1                                          tls.ldif
dn: cn=config
changetype: modify
replace: olcTLSCertificateFile
olcTLSCertificateFile: /etc/openldap/certs/ldap.crt
-
replace: olcTLSCertificateKeyFile
olcTLSCertificateKeyFile: /etc/openldap/certs/ldap.key
```

We mapped he IP address 10.0.0.68 to the name host68.datalab so that the SSL certificate's Common Name (CN) would match the connection string. We did the same for 10.0.0.67.



On the server we created our own CA (myca.crt). This acts as the "Master Key" that will be used to verify the server's identity.

```
soukup@localhost:~/task5$ mkdir ~/certs && cd ~/certs
soukup@localhost:~/certs$ openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout myca.key -out myca.crt -subj "/
O=HVL/OU=datalab/CN=MyCA"
..++++++++++++++++++++++++++++++++++++++++*..+.......+.....+...+....+......+..+....+.....+......+....+...+.....+.+..++
++++++++++++++++++++++++++++++++++++++++*..+...+.........+......+...+....+......+..............+...+...+....+.....+.....
..+...+......+......+.........+.....+....+...+.........+.+...+.........+.......+..+.+...+......+...+.........+...+......
......+....+...+......+.....+.+.....+...+............+...+...+...+.......+........+..........+......+.....+...+......
....+...+.........................................+............+....+.....+.+...+.........+...+..+.+...........+....+..+
..........+..+.+.........+...+..+.......+......+..+.+..+..........+........+.+.....................+.+.............
.+.......+.........+......+.....+....+......+.........+...........+...+...+...+......+....+.....+.........+...+......
.+.....++++++
..+........+..........+..+......+....+.........+..+.........+..........+...+......+......+.........+..+.......+..++++
++++++++++++++++++++++++++++++++++*.........+..+....+...+..+.+.........+.....+....+..+...+....+......++++++++++++++++
++++++++++++++++++++++*..+.........+.........+.......+.....+....+..+.........+....++++++
-----
soukup@localhost:~/certs$ openssl req -nodes -newkey rsa:2048 -keyout ldap.key -out ldap.csr -subj "/O=HVL/OU=datalab
/CN=host68.datalab"
.....+......+.....+.++++++++++++++++++++++++++++++++++++++++*...++++++++++++++++++++++++++++++++++++++++*..+.+..+......
.+........++++++
.++++++++++++++++++++++++++++++++++++++++*.........++++++++++++++++++++++++++++++++++++++++*.....+..+............+....+
......++++++
-----
soukup@localhost:~/certs$ openssl x509 -req -in ldap.csr -CA myca.crt -CAkey myca.key -CAcreateserial -out ldap.crt -
days 365
Certificate request self-signature ok
subject=O=HVL, OU=datalab, CN=host68.datalab
```

This clients Certificate Authority (CA):

```
ffeilke@host67:/etc/openldap/certs$ sudo nano myca.crt
ffeilke@host67:/etc/openldap/certs$ cat myca.crt
-----BEGIN CERTIFICATE-----
MIIDPzCCAiegAwIBAgIUNVRJGxYyeo0QhrfsKTphvf0DdLcwDQYJKoZIhvcNAQEL
BQAwLzEMMAoGA1UECgwDSFZMMRAwDgYDVQQLDAdkYXRhbGFiMQ0wCwYDVQQDDARN
eUNBMB4XDTI2MDMyMzEyMjIyN1oXDTI3MDMyMzEyMjIyN1owLzEMMAoGA1UECgwD
SFZMMRAwDgYDVQQLDAdkYXRhbGFiMQ0wCwYDVQQDDARNeUNBMIIBIjANBgkqhkiG
9w0BAQEFAAOCAQ8AMIIBCgKCAQEAuaHn7Fuviz2NcLXcs/knVSm7o7CQ9xAJowE3
wx4SrWWizutLVnifITIZlyb0xz71pu1XPaYXdJtN4WJJokBb7u4UDDZ0M6f7vdt1
1BpZXWTcoupQXzJSjlZJ603FnrbB7xg7bm8wfrXlkO3H9nyRkeUhE+UTapnoc1q8
GFOONn338lx9fm0L/o42TgG8vyqPpSp3TMnVIKP5VkLA3PwGQCxMvzRDVRYAwkFT
ijtdfUXZZ/0xcODrL+iZnYnMIPKTOHwK3eQ/Cvs1FlbfGIjYJRzybhgMgsqPA1zp
HPlj8wY1N/0yffTqrnAuZ2oUPtKvxR9w9h2q3r3aenpDCkpeswIDAQABo1MwUTAd
BgNVHQ4EFgQUX3lsWuMzQWB3NCrU7HgtbXOjksYwHwYDVR0jBBgwFoAUX3lsWuMz
QWB3NCrU7HgtbXOjksYwDwYDVR0TAQH/BAUwAwEB/zANBgkqhkiG9w0BAQsFAAOC
AQEAYdjLvyqQ2V87uOE4jg+rMhrc33zM/b0w64L7uTg3/4Xf/upTi7Xi8MXgQCP7
HnBAu+bYCllxGKfONJ5VNkB1BdDdSoBIrainsl/Hk8kmts5WqVIw51dV4u7UWHX9
QuB5PhAu93yW9gfsleMi5wjXQQO3YL+I7WvwQSsLWYajeoXqcTwXKfdxw4X3XWXZ
FNfCisbvaNrEVxLRwn3RdM2ZFkKlOf+tw4C695eu/v0QJOE61becUm+l3dwaEoBy
X0Nu79pE2xkcqkqrM26L4XIu4Hoczb5U9Ia7Lj+1033J6r0wJE/FjiRBDXkOXlDs
pACpnI7dyRmhviEJD88VXYc5xA==
-----END CERTIFICATE-----
ffeilke@host67:/etc/openldap/certs$
```

The client will use the myca.crt file we just created and the servers certificate that the conection is save.

LDAP Cert Configuration:

We created /etc/openldap/certs as a standard, secure location for certificate files. Then we move the certificates created in the previous step into the file. We also changed the ownership of the files to the ldap user. With chmod 600 we make sure, that only the LDAP service can read the private key, protecting the server from being impersonated.

```
soukup@localhost:~/certs$ sudo mkdir -p /etc/openldap/certs
soukup@localhost:~/certs$ sudo cp myca.crt ldap.crt ldap.key /etc/openldap/certs/
soukup@localhost:~/certs$ sudo chown ldap:ldap /etc/openldap/certs/ldap.*
soukup@localhost:~/certs$ sudo chmod 600 /etc/openldap/certs/ldap.key
```

We told the LDAP server to use the specific identity files we created by modifying the cn=config tree.

```
 GNU nano 8.1                                                tls.ldif
dn: cn=config
changetype: modify
replace: olcTLSCertificateFile
olcTLSCertificateFile: /etc/openldap/certs/ldap.crt
-
replace: olcTLSCertificateKeyFile
olcTLSCertificateKeyFile: /etc/openldap/certs/ldap.key
```

After the LDAP server was configured to use SSL/TLS, the network firewall needed to be updated to allow traffic on the secure port.

```
soukup@localhost:~/certs$ sudo firewall-cmd --add-service=ldaps --permanent
success
soukup@localhost:~/certs$ sudo firewall-cmd --reload
success
soukup@localhost:~/certs$
```

SSSD Update:

```
 GNU nano 8.1                                  /etc/sssd/sssd.conf
[sssd]
services = nss, pam
domains = LDAP

[domain/LDAP]
id_provider = ldap
auth_provider = ldap
ldap_uri = ldaps://host68.datalab
ldap_search_base = dc=h68,dc=dat151

ldap_chpass_update_last_change = true
ldap_tls_cacert = /etc/openldap/certs/myca.crt
cache_credentials = true
enumerate = true
```

This forces the client to connect via port 636 and initiate an SSL/TLS handshake immediately.And it tells SSSD to use the myca.crt file we created earlyer to verify the server's identity.

Task 4: Kerberos

Configure a host as a KDC (Key Distribution Center) and also use it as a Kerberos client to authenticate SSH logins.

KDC Configuration

Configure KDC, create the database, and enable services (`kadmin`, `krb5kdc`). Create the KDC database with a secure password.

At first we installed the krb5-server and the krb5-workstation.

```
soukup@localhost:~$ sudo dnf install krb5-server krb5-workstation
[sudo] password for soukup:
Last metadata expiration check: 1:03:32 ago on Mon 23 Mar 2026 12:34:08 PM CET.
Dependencies resolved.
================================================================================
 Package                Architecture          Version                 Repository             Size
================================================================================
Installing:
 krb5-server            x86_64                1.21.3-8.el10_0         baseos                298 k
 krb5-workstation       x86_64                1.21.3-8.el10_0         baseos                402 k
Installing dependencies:
 krb5-pkinit            x86_64                1.21.3-8.el10_0         baseos                 60 k
 libev                  x86_64                4.33-14.el10            baseos                 50 k
 libkadm5               x86_64                1.21.3-8.el10_0         baseos                 76 k
 libverto-libev         x86_64                0.3.2-10.el10           baseos                 13 k

Transaction Summary
================================================================================
```

We Configured the Kerberos Realm to use our Domain DATALAB.HVL.

```
  GNU nano 8.1                                  /var/kerberos/krb5kdc/kdc.conf
[libdefaults]
# Allow RC4 HMAC-MD5 for session keys (see CVE-2022-37966)
#allow_rc4 = true

[kdcdefaults]
 kdc_ports = 88
 kdc_tcp_ports = 88

[realms]
 DATALAB.HVL = {
  #master_key_type = aes256-cts
  acl_file = /var/kerberos/krb5kdc/kadm5.acl
  dict_file = /usr/share/dict/words
  admin_keytab = /var/kerberos/krb5kdc/kadm5.keytab
  supported_enctypes = aes256-cts:normal aes128-cts:normal
 }
```

```
  GNU nano 8.1                                          /etc/krb5.conf
# To opt out of the system crypto-policies configuration of krb5, remove the
# symlink at /etc/krb5.conf.d/crypto-policies which will not be recreated.
includedir /etc/krb5.conf.d/

[logging]
    default = FILE:/var/log/krb5libs.log
    kdc = FILE:/var/log/krb5kdc.log
    admin_server = FILE:/var/log/kadmind.log

[libdefaults]
    dns_lookup_realm = false
    ticket_lifetime = 24h
    renew_lifetime = 7d
    forwardable = true
    rdns = false
    pkinit_anchors = FILE:/etc/pki/tls/certs/ca-bundle.crt
    spake_preauth_groups = edwards25519
    dns_canonicalize_hostname = fallback
    qualify_shortname = ""
    default_realm = DATALAB.HVL
    default_ccache_name = KEYRING:persistent:%{uid}

[realms]
 DATALAB.HVL = {
  kdc = host68.datalab
  admin_server = host68.datalab
 }

[domain_realm]
 .datalab = DATALAB.HVL
 datalab = DATALAB.HVL
```

Then we createt our database

```
soukup@localhost:~$ sudo kdb5_util create -s -r DATALAB.HVL
Initializing database '/var/kerberos/krb5kdc/principal' for realm 'DATALAB.HVL',
master key name 'K/M@DATALAB.HVL'
You will be prompted for the database Master Password.
It is important that you NOT FORGET this password.
Enter KDC database master key:
Re-enter KDC database master key to verify:
```

-s flag creates a "stash file" so the KDC can start automatically without you typing the master password every time it boots. -r flag allows us to explicitly specify the name of the Kerberos realm.

We need the firewall to open the ports that kerberus and kadmin are mapped to.

```
soukup@localhost:~$ sudo firewall-cmd --add-service=kerberos --add-service=kadmin --permanent
success
soukup@localhost:~$ sudo firewall-cmd --reload
success
```

In the Access Control List we give the admin full permission to add, delete, or modify other users (principals).

```
 GNU nano 8.1                                  /var/kerberos/krb5kdc/kadm5.acl
*/admin@DATALAB.HVL     *
```

We use this command to edit the Access Control List:

```
soukup@localhost:~$ sudo nano /var/kerberos/krb5kdc/kadm5.acl
soukup@localhost:~$ sudo systemctl restart kadmin
```

Principals

Create necessary principals (User, Admin, Host, Service). Since only root can run kadmin.local, and the default principal of root is "root/admin", create an administrator principal "root/admin".

Principals Created:

- User: testuser@DATALAB.HVL
- Admin:root/admin@DATALAB.HVL
- Host:host/hostname68.datalab@DATALAB.HVL)

Commands:

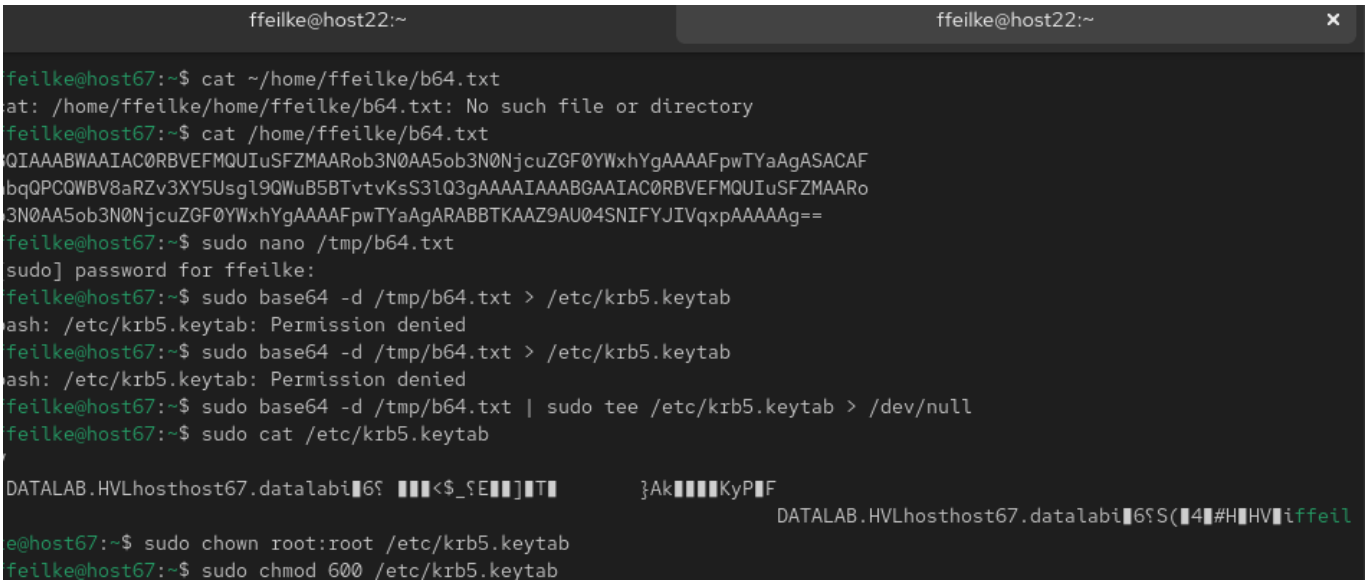
```
soukup@localhost:~$ sudo kadmin.local
Authenticating as principal root/admin@DATALAB.HVL with password.
kadmin.local:  addprinc root/admin
No policy specified for root/admin@DATALAB.HVL; defaulting to no policy
Enter password for principal "root/admin@DATALAB.HVL":
Re-enter password for principal "root/admin@DATALAB.HVL":
Principal "root/admin@DATALAB.HVL" created.
kadmin.local:  addprinc testuser
No policy specified for testuser@DATALAB.HVL; defaulting to no policy
Enter password for principal "testuser@DATALAB.HVL":
Re-enter password for principal "testuser@DATALAB.HVL":
Principal "testuser@DATALAB.HVL" created.
kadmin.local:  addprinc -randkey host/host68.datalab
No policy specified for host/host68.datalab@DATALAB.HVL; defaulting to no policy
Principal "host/host68.datalab@DATALAB.HVL" created.
kadmin.local:  addprinc -randkey host/host67.datalab
No policy specified for host/host67.datalab@DATALAB.HVL; defaulting to no policy
Principal "host/host67.datalab@DATALAB.HVL" created.
kadmin.local:  ktadd host/host68.datalab
Entry for principal host/host68.datalab with kvno 2, encryption type aes256-cts-hmac-sha1-96 added to keytab FILE:/et
c/krb5.keytab.
Entry for principal host/host68.datalab with kvno 2, encryption type aes128-cts-hmac-sha1-96 added to keytab FILE:/et
c/krb5.keytab.
kadmin.local:  ktadd -k /tmp/host67.keytab host/host67.datalab
Entry for principal host/host67.datalab with kvno 2, encryption type aes256-cts-hmac-sha1-96 added to keytab WRFILE:/
tmp/host67.keytab.
Entry for principal host/host67.datalab with kvno 2, encryption type aes128-cts-hmac-sha1-96 added to keytab WRFILE:/
tmp/host67.keytab.
kadmin.local:  quit
soukup@localhost:~$
```

With ktadd we export the Host Principal into a keytab file. This is necessary for an authentification without a password.

SSH with Kerberos

Configure SSH to use Kerberos. Verify login in both directions.

Configuration Changes:



```
soukup@localhost:~$ sudo systemctl status krb5kdc
○ krb5kdc.service - Kerberos 5 KDC
     Loaded: loaded (/usr/lib/systemd/system/krb5kdc.service; disabled; preset: disabled)
     Active: inactive (dead)
soukup@localhost:~$ sudo systemctl enable --now krb5kdc kadmin
Created symlink '/etc/systemd/system/multi-user.target.wants/krb5kdc.service' → '/usr/lib/systemd/system/krb5kdc.serv
ice'.
Created symlink '/etc/systemd/system/multi-user.target.wants/kadmin.service' → '/usr/lib/systemd/system/kadmin.servic
e'.
soukup@localhost:~$ sudo systemctl status krb5kdc
● krb5kdc.service - Kerberos 5 KDC
     Loaded: loaded (/usr/lib/systemd/system/krb5kdc.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-03-23 14:10:51 CET; 6s ago
 Invocation: fd8450687d454a489139c670a85dd4a4
    Process: 10148 ExecStart=/usr/sbin/krb5kdc -P /run/krb5kdc.pid $KRB5KDC_ARGS (code=exited, status=0/SUCCESS)
   Main PID: 10152 (krb5kdc)
      Tasks: 1 (limit: 46648)
     Memory: 4.9M (peak: 5.3M)
        CPU: 20ms
     CGroup: /system.slice/krb5kdc.service
             └─10152 /usr/sbin/krb5kdc -P /run/krb5kdc.pid

Mar 23 14:10:51 host68.datalab systemd[1]: Starting krb5kdc.service - Kerberos 5 KDC...
Mar 23 14:10:51 host68.datalab systemd[1]: Started krb5kdc.service - Kerberos 5 KDC.
soukup@localhost:~$ sudo klist -k /etc/krb5.keytab
Keytab name: FILE:/etc/krb5.keytab
KVNO Principal
---- --------------------------------------------------------------------------
   2 host/host68.datalab@DATALAB.HVL
   2 host/host68.datalab@DATALAB.HVL
soukup@localhost:~$ sudo sshd -T | grep gssapiauthentication
gssapiauthentication yes
soukup@localhost:~$ getent passwd testuser
soukup@localhost:~$ sudo useradd -u 2000 testuser
[sudo] password for soukup:
soukup@localhost:~$ getent passwd testuser
testuser:x:2000:2000::/home/testuser:/bin/bash
```


```
soukup@localhost:~$ getent passwd testuser
soukup@localhost:~$ sudo useradd -u 2000 testuser
[sudo] password for soukup:
soukup@localhost:~$ getent passwd testuser
testuser:x:2000:2000::/home/testuser:/bin/bash
```


Verification:

```
testuser@host67:~$ ssh  host68.datalab

1 device has a firmware upgrade available.
Run `fwupdmgr get-upgrades` for more information.

Last failed login: Mon Mar 23 14:20:07 CET 2026 from 10.0.0.67 on ssh:notty
There were 2 failed login attempts since the last successful login.
testuser@host68:~$
```